



An Evaluation of Massively Parallel Algorithms for DFA Minimization

Seminar Presentation

By: Gokul Krishna | Reg. No: MUT22CS066

Guide: Dr. Sheena Kurian K, Asst. Prof.

Presentation Roadmap: Navigating the Evaluation

1

Motivation & Background

Understanding the need for parallel DFA minimization.

2

Literature Review

Key foundational and modern works in the field.

3

Methodology & Algorithms

In-depth look at the four parallel approaches evaluated.

4

Empirical Analysis

Experimental findings, performance trade-offs, and practical conclusions.

5

Future Scope & References

Potential research directions and sources.

The Problem and Goal: Why Parallelize DFA Minimization?

- The Problem: Compute a **Minimal Deterministic Finite Automaton (DFA)** equivalent to a given DFA by merging indistinguishable states.
- Importance: Crucial for applications like model checking, protocol analysis, regular expression engines, and compiler optimization.
- The Challenge: Classical algorithms are sequential, leading to slow performance on **very large DFAs** (often encountered in state-space exploration).
- GPU Opportunity: Parallel GPU architectures promise significant speedup but introduce resource trade-offs (memory constraints, synchronization costs).

Goal of the Base Paper

To implement and empirically compare **four parallel GPU algorithms** for DFA minimization, evaluating their practical performance against theoretical expectations.



Foundational Concepts and Relevance

DFA Fundamentals

A DFA is defined by states (Q), an alphabet (Σ), a transition function (δ), and accepting states (F).

Minimization Core

Finding the **coarsest partition** of Q such that all states within the same block are equivalent (i.e., generate the same language).

Partition Refinement

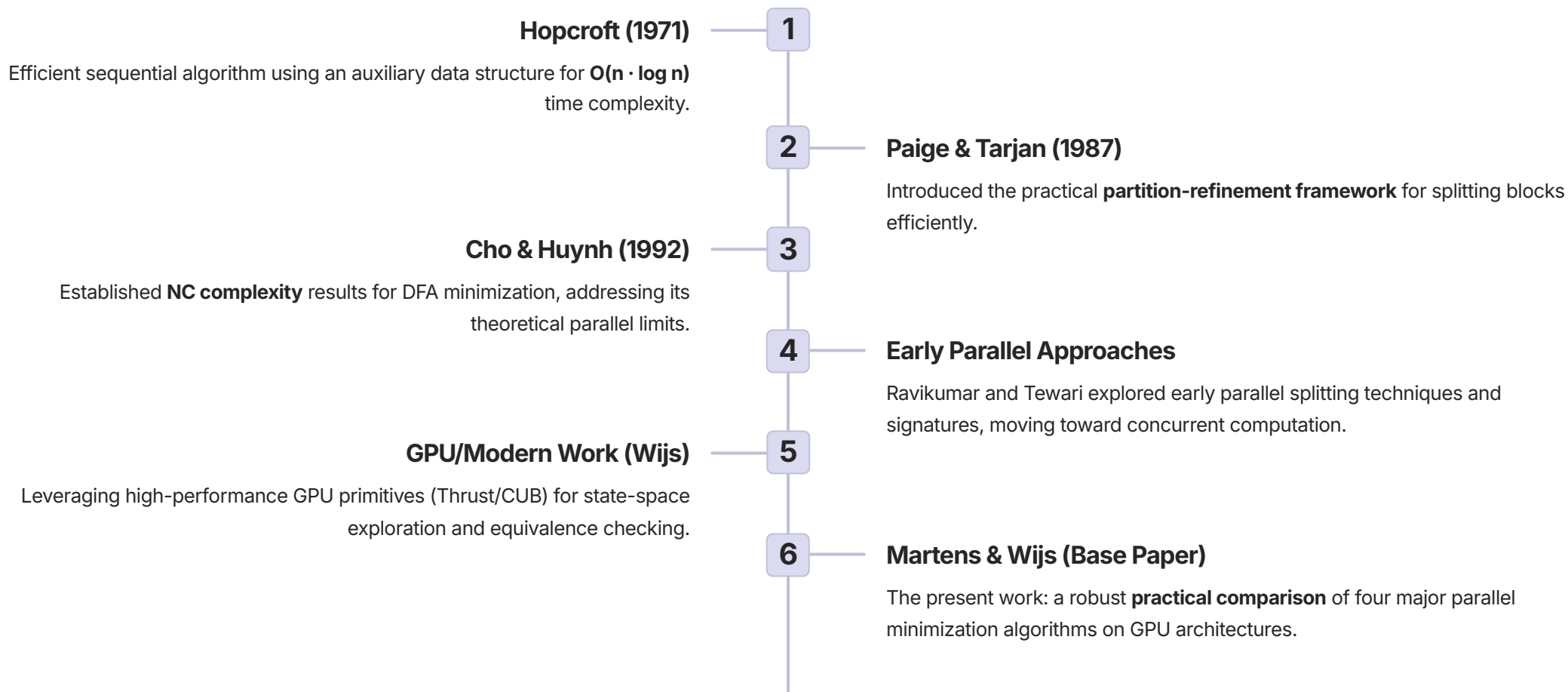
The iterative process that starts with coarse blocks (Accepting / Non-accepting) and **splits** blocks whose members exhibit different behavior under transitions.

Parallel Trade-Off

Theoretically fast **NC (Polylog time)** algorithms demand massive resources. GPUs offer parallel processing but are constrained by **limited memory** and synchronization overheads.

Key Literature on DFA Minimization

The foundation of this research spans classical sequential methods, theoretical parallel models, and modern GPU implementations.



Four Massively Parallel Algorithms

The study implemented and compared four distinct approaches on the GPU platform.



trans

Pairwise Reachability (Transitive Closure)



naivePR

Naive Parallel Partition Refinement



sortPR

Signature-based Refinement with Parallel Sort



transPR

Doubling + Partition Refinement Hybrid

Algorithm

trans: Pairwise Reachability

- Build nodes for every state pair $(p, q) \in Q \times Q$
- Mark trivial unequal pairs (accepting XOR non-accepting).
- For each symbol a : add edge $(p, q) \rightarrow (\delta(p, a), \delta(q, a))$
- Compute transitive closure / reachability from marked nodes.
- Any pair that reaches a marked node is unequal.
- Unmarked pairs represent equivalent states $\rightarrow$ group into blocks.
- Theoretically polylog depth; huge $O(n^2)$ node overhead.

❏ Memory Overhead: Quickly **memory-bound** due to **$O(n^2)$** nodes and potentially massive edge sets, despite polylog depth.

Algorithm

naivePR — Naive parallel partition refinement

- Initialize blocks: accepting vs non-accepting.
- Process each block independently in parallel.
- Elect a leader state for the block (representative).
- Each state compares its transition-blocks to leader's.
- States that differ are marked to split off.
- Compact marked states and assign new block IDs.
- Repeat rounds until no more splits occur.

❏ Per-round work $O(n.k)$ iterations may be large (worst-case many rounds). Memory: linear in n .

Algorithm

sortPR — Signature-based refinement with parallel sort

- Compute signature for every state in parallel.
- Signature = (currentBlock, dest-block IDs per symbol).
- Pack signature into fixed-size sortable key if possible.
- Parallel-sort states by signature (fast on GPU).
- Adjacent-diff groups identical signatures into clusters.
- Assign new block IDs via prefix-scan (one pass).
- Repeat until block IDs no longer change.

Quick complexity / resource:

Per-round cost dominated by parallel sort; iterations usually few. Memory: linear but higher constants (sorting buffers).

Algorithm

transPR

- Precompute long-step transitions by doubling: $\delta, \delta^2, \delta^4, \dots$
- Extend alphabet with long-step symbols ($\Sigma \rightarrow \Sigma \cup \Sigma_T$).
- Build signatures using these extended transitions.
- Run partition refinement (naivePR or sortPR) on extended DFA.
- Captures long behavior in few rounds (fewer splits).
- Overhead: extra memory & compute for doubling tables.
- Best for unary/long-chain DFAs (e.g., Fibonacci family).

Quick complexity / resource:

Extra prework $O(n \log L)$ and memory for δ^{2^i} ; reduces refinement rounds significantly for chain-like DFAs.

Experimental Findings and Practical Recommendation

Practicality Over Theory



sortPR

General-purpose winner on real-world VLTS benchmarks due to large, fast splits per iteration.



transPR

Excels on special cases: **unary or long-chain DFAs** (e.g., Fibonacci family) due to reduced iteration count.



naivePR

Useful for **memory-constrained** scenarios; cheaper iterations can outperform sortPR on specific structured inputs.



trans

Deemed **impractical** for massive parallelization due to exponential memory/time explosion in tested environments.

Final Pick: **sortPR** is the robust default. Use **transPR** for chain-like inputs and **naivePR** as a memory-safe fallback.

- Theoretical `trans` (pairwise reachability) is **impractical**: memory/time explode.
- Partition-refinement methods are **far more practical** on GPUs.
- **sortPR** wins on real-world VLTS benchmarks — fewer iterations, large splits per iteration.
- **transPR** (partial transitive closure + PR) excels on **unary / long-chain DFAs** (Fibonacci family).
- **naivePR** is memory-light and can outperform sortPR on some structured inputs (Fibonacci/bit-splitter) due to cheaper iterations.
- Practical pick: **default = sortPR**; use **transPR** when input shows long same-symbol chains; fallback **naivePR** for memory-tight cases

Conclusion

- Theoretically optimal **transitive closure** is resource-prohibitive on modern GPUs.
- **Partition Refinement** remains the practical family for GPU-accelerated DFA minimization.
- **sortPR** is the recommended general-purpose algorithm for most real-world systems.
- **transPR** provides necessary specialization for significant speedups on long-chain DFAs.
- Practical roadmap: implement adaptive pipeline — *pre-analyze shape* → *choose algorithm* → *fall back to memory-safe method*
- The future lies in an **adaptive pipeline** that selects the optimal algorithm based on input structure.

Future Scope

- **Work-efficient reachability:** explore GPU algorithms with near-linear work and sublinear depth for transitive steps.
- **Adaptive hybrid model:** automatically choose between sortPR, naivePR, or transPR based on DFA structure.
- **Optimized memory use:** compress transition tables and signatures to handle larger DFAs on limited GPU memory.
- **CPU–GPU collaboration:** design pipelines where CPU handles control flow and GPU handles heavy parallel kernels.
- **Distributed GPU scaling:** extend implementation to multi-GPU or cluster environments for massive DFAs.
- **Real-world testing:** evaluate on larger VLTS models and industrial automata to validate scalability.
- **AI-driven tuning:** use machine learning to predict the best algorithm and parameters for a given input.

References

Base Paper

[1] J. Martens and A. Wijs, *"An Evaluation of Massively Parallel Algorithms for DFA Minimization,"* in *Proc. 15th Int. Symposium on Games, Automata, Logics, and Formal Verification (GandALF 2024)*, 2024.

Reference Papers

[2] J. E. Hopcroft, *"An $n \log n$ Algorithm for Minimizing States in a Finite Automaton,"* in *Theory of Machines and Computations*, Academic Press, 1971.

[3] R. Paige and R. E. Tarjan, *"Three Partition Refinement Algorithms,"* *SIAM Journal on Computing*, vol. 16, no. 6, pp. 973–989, 1987.

[4] A. Tewari, M. Ravikumar, and N. R. Arora, *"Parallel DFA Minimization,"* in *Proc. High-Performance Computing (HiPC)*, 2002.

[5] A. J. Wijs, *"GPU-Accelerated Bisimilarity Checking,"* in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2015.

Web Sources

[8] https://en.wikipedia.org/wiki/Deterministic_finite_automaton

[9] <https://developer.nvidia.com/thrust>

Thank you